Decision Trees & Ensemble Methods

A Tutor-Style Study Report — Continuous Features, Regression Trees, Bagging, Random Forests & XGBoost

ML Specialisation — Source Report: *Decision Trees and Ensemble Methods*
Structure: What We Have • What We Really Want • What We Will Use • Other Ways to Solve It • The Problem We Face • How to Solve It • Summary

This report reorganises the source material on decision trees for continuous features, regression trees, and tree ensembles (Bagging, Random Forests, XGBoost) into a structured tutor walkthrough, with worked equations, comparison tables, and supporting charts.

1 What We Have

Our starting point is a cat-vs-dog adoption dataset that, so far, has only used a **discrete** feature: Ear Shape (pointy / floppy). We now extend it with:

- A **continuous feature**: *Weight* (a numerical value on a spectrum, not a category).
- 10 labelled training examples (cats vs. dogs) with known weights.
- A parallel dataset where the target itself is continuous — the animal's *weight* — used to motivate **regression trees**.
- A single decision tree that we know is **sensitive**: changing just one training example can flip the chosen root split entirely (Ear Shape → Whiskers).

⚠ ***Assumption / Caution:*** *We assume the dataset is small and clean (no missing weights, no mislabeled classes) purely for pedagogical clarity — real datasets need cleaning first.*

2 What We Really Want

Four escalating goals drive this material, each building on the last:

- **Goal A:** Let a decision tree split on a continuous feature (Weight) instead of only discrete ones.
- **Goal B:** Let a tree predict a continuous *target* (regression), not just a class label.
- **Goal C:** Make predictions **robust** to small changes in the training data by combining many trees into an ensemble.
- **Goal D:** Choose, with justification, between tree ensembles (XGBoost) and neural networks for a real structured-data problem (e.g. predicting home prices).

3 What We Will Use

3.1 Thresholding continuous features

For M sorted examples, there are exactly M−1 candidate midpoint thresholds. Each candidate is scored with **Information Gain**:

$$IG = H(p_{root}) - [\, w_{left} \cdot H(p_{left}) + w_{right} \cdot H(p_{right}) \,]$$

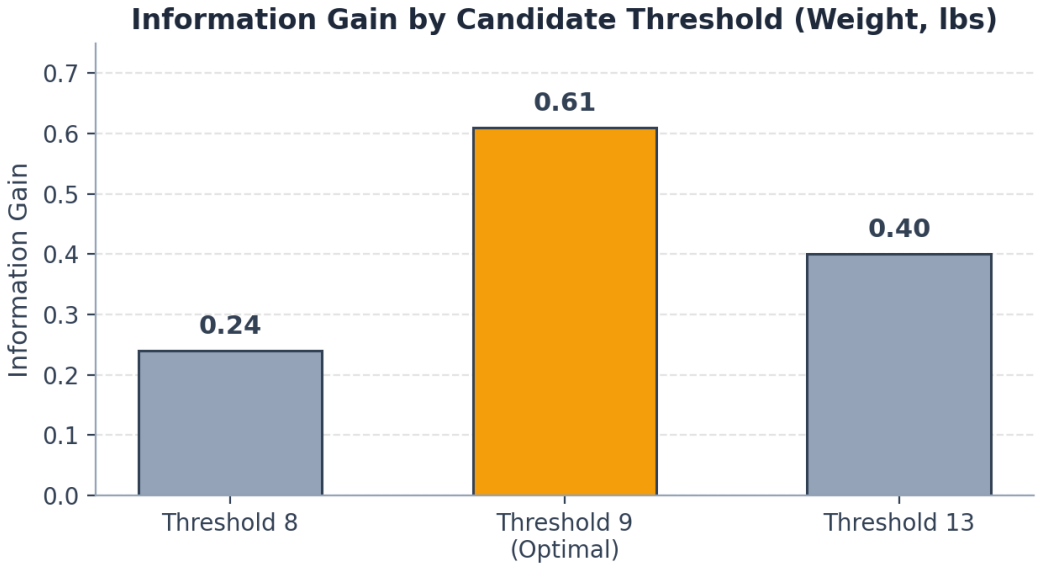


Figure 3.1 — Information Gain for three candidate weight thresholds; Threshold 9 wins.

Threshold	Information Gain	Selected?
8 lbs	0.24	No
9 lbs	0.61	Yes — optimal
13 lbs	0.40	No

3.2 Variance reduction (regression trees)

When the target Y is continuous, a leaf predicts the **mean** of Y over its training examples, and splits are scored by how much they shrink the spread of Y:

$$\Delta Variance = Var(root) - [\, w_{left} \cdot Var(left) + w_{right} \cdot Var(right) \,]$$

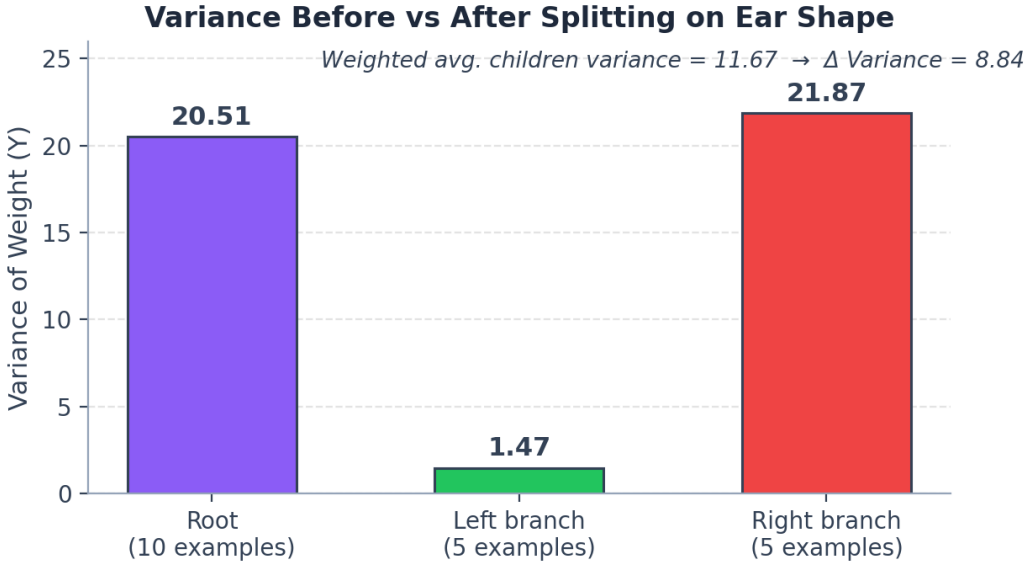


Figure 3.2 — Root variance (20.51) drops to a weighted average of 11.67 after splitting on Ear Shape → ΔVariance = 8.84.

3.3 Sampling with replacement (Bagging)

Each tree in an ensemble is trained on a new dataset of size M, drawn **with replacement** from the original M examples — some examples repeat, others are left out entirely.

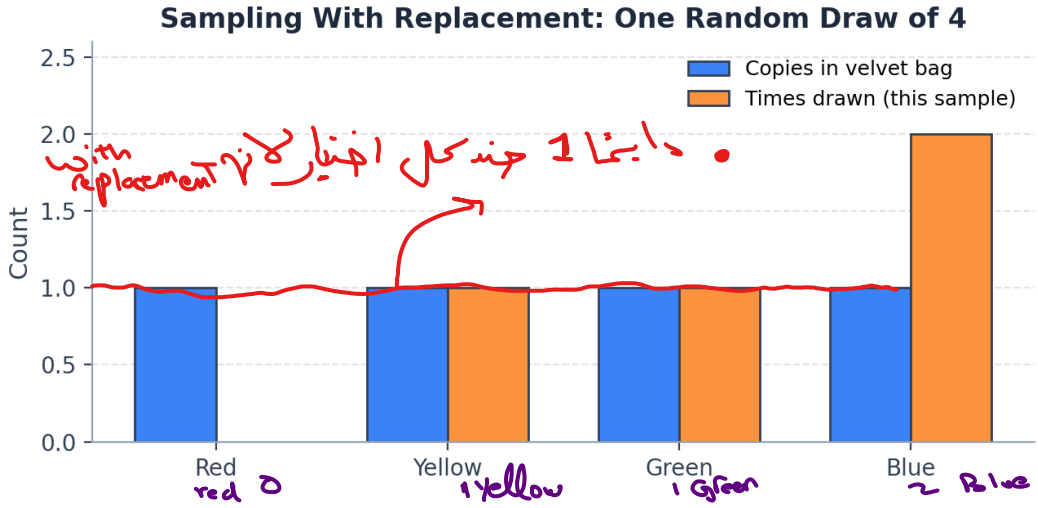


Figure 3.3 — The velvet-bag analogy: one draw of 4 tokens with replacement (illustrative sample: Green, Yellow, Blue, Blue).

3.4 Random Forest & XGBoost mechanics

- **Bagged trees:** repeat sampling-with-replacement B times, B typically between 64 and 128.
- **Random Forest:** additionally restrict each split to a random subset of K features out of N, with $K \approx \sqrt{N}$.

- **XGBoost:** instead of uniform resampling, *re-weight* misclassified examples so the next tree focuses on the current model's errors ("deliberate practice").

```python
from xgboost import XGBClassifier, XGBRegressor

# Classification
model_class = XGBClassifier()
model_class.fit(X_train, y_train)
predictions = model_class.predict(X_test)

# Regression
model_reg = XGBRegressor()
model_reg.fit(X_train, y_train)
```

⚠ ***Assumption / Caution:*** *K = √N is a rule of thumb; its benefit is most visible with dozens/hundreds of features and is negligible on a toy 3-feature dataset.*

4 Other Ways to Solve It

The same prediction problem can be approached with different methods, each governed by different underlying assumptions ('laws') and limits:

Method	Governing Idea	Best When	Key Limit
Single Decision Tree	Greedy IG / variance splits	Quick, interpretable baseline	High variance — unstable to small data changes
Bagged Trees	Bootstrap aggregation + vote	Reducing variance cheaply	All features considered at every split → correlated trees
Random Forest	Bagging + feature subsampling	Many (dozens+) features	Little benefit on small feature sets
XGBoost (Boosting)	Sequential error-weighting	Structured / tabular data, competitions	Less natural for unstructured data (images, audio, text)
Neural Network	Learned distributed representations	Unstructured or mixed data, transfer learning	Slower to train; large models need more data/compute

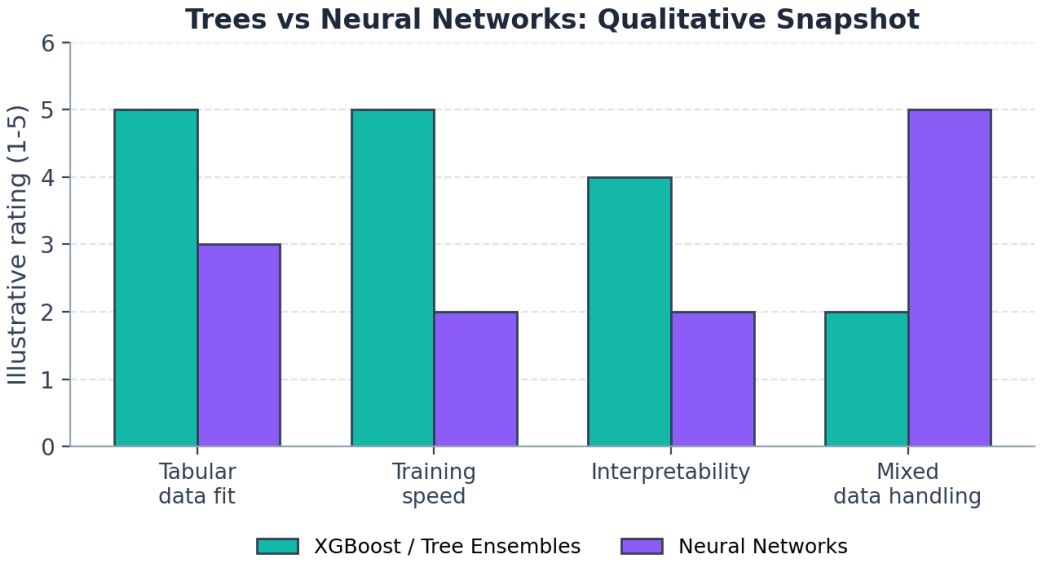


Figure 4.1 — Illustrative, qualitative snapshot summarising the source comparison table (not measured benchmarks).

⚠ ***Assumption / Caution:*** *The chart above translates qualitative labels ('High', 'Limited', 'Excellent') from the source table into an illustrative 1–5 scale purely to visualise relative strengths — it is an approximation, not a benchmark result.*

5 The Problem We Face

- **Instability:** A single tree's root split can change completely (Ear Shape → Whiskers) from just one modified training example — predictions are not trustworthy in isolation.
- **Correlated trees in plain Bagging:** because every tree can still pick from all N features, bagged trees tend to converge on similar splits, limiting diversity.
- **Slow learning on errors:** plain bagging treats every example equally, so persistently hard examples don't get extra attention.
- **Model-choice risk:** picking a neural network for a small, structured/tabular dataset (or a tree ensemble for raw images) wastes training speed and accuracy potential.

6 How to Solve It

6.1 Algorithm: choosing a continuous-feature threshold

1. Sort all M examples by the continuous feature.
2. Compute the M−1 midpoints between adjacent sorted values as candidate thresholds.
3. For each candidate, split the data and compute Information Gain (classification) or ΔVariance (regression).

4. Select the threshold that maximises the score; use it as the split rule for that node.

6.2 Algorithm: building a Random Forest

1. For b = 1 to B (B ≈ 64–128): draw a bootstrap sample of size M with replacement.
2. At every node of tree b, randomly select K ≈ √N features and choose the best split only among them.
3. Grow tree b fully (or to a stopping rule) on its bootstrap sample.
4. At prediction time, run the example through all B trees and combine by majority vote (classification) or averaging (regression).

6.3 Algorithm: boosting with XGBoost

1. Train an initial tree on the (equally weighted) training data.
2. Identify examples the tree gets wrong and increase their weight.
3. Train the next tree with more focus on the up-weighted, difficult examples.
4. Repeat, then combine all trees' outputs; regularisation is applied internally to limit overfitting.

6.4 Decision rule: trees vs. neural networks

Worked scenario: 10,000 homes, columns for Square Footage, Zip Code, Price → predict price for new listings.

Structured / tabular data → prefer XGBoost (faster iteration, strong default performance)

Because the data is structured/tabular and fast iteration matters, XGBoost is the recommended choice over a neural network here; a neural network would be preferred only if the inputs were unstructured (images/audio/text) or mixed with unstructured signals.

7 Summary

Foundational pillars

- Information Gain and Variance Reduction are the two core metrics for choosing any split.
- Sampling with replacement (bootstrap) is the mechanism behind Bagging and Random Forests.
- Boosting's core intuition is sequential, weighted focus on mistakes ('deliberate practice').

Supplementary details (library-abstracted)

- The exact probability-weight math inside boosting is handled internally by XGBoost.
- Regularisation exists and matters, but its precise mathematical derivation is an advanced implementation detail rarely hand-derived in practice.

Takeaway: for tabular/structured prediction problems, start with a single tree to understand the data, move to a Random Forest or XGBoost for robustness and accuracy, and reserve neural networks for unstructured or mixed data where their representation-learning strength pays off.